



SEGURIDAD UNIVERSITARIA

Documentación Completa

Sistema de Seguridad Universitaria

*Plataforma integral de control de accesos, reporte de incidentes
y gestión de seguridad para el campus universitario*

Autora:	Gabriela Cordoba Gonzalez
Versión:	Edición Completa
Año:	2026
Tipo:	Documentación técnica y funcional
Stack:	React.js · Node.js · MongoDB · Nginx
Arquitectura:	Microservicios independientes
Metodología:	Scrum Ágil — 5 Sprints
Clasificación:	Documentación interna

Resumen Ejecutivo

El **Sistema de Seguridad Universitaria (SSU)** es una plataforma web de gestión integral desarrollada con el propósito de digitalizar y centralizar todos los procesos relacionados con la seguridad del campus universitario. La plataforma surge como respuesta directa a la ausencia total de herramientas tecnológicas en los procedimientos de control de accesos, registro de incidentes y gestión de objetos perdidos en la institución.

El sistema ha sido concebido bajo una **arquitectura de microservicios** con Node.js y MongoDB, un frontend moderno en React.js con Tailwind CSS, y desplegado en producción con Nginx como proxy inverso y SSL mediante Let's Encrypt. La metodología de desarrollo adoptada es **Scrum Ágil**, con cinco sprints de dos semanas cada uno, completando un total de 210 horas de trabajo distribuidas entre cuatro desarrolladores.

Indicadores Clave del Proyecto

Módulos: 6	Roles: 4	Sprints: 5	Semanas: 13
Horas: 210 h	Devs: 4	Presupuesto: \$15.180.000 COP	Microservicios: 5

Los beneficios esperados incluyen el registro digital del **100%** de los accesos al campus, una reducción del **80%** en el tiempo de respuesta ante incidentes, la gestión efectiva del **95%** de los objetos perdidos reportados, y disponibilidad del sistema **24/7** con mecanismos de recuperación ante fallos.

Índice

Resumen Ejecutivo	2
1 Problemática	9
1.1 Contexto Institucional	9
1.2 Situación Actual — Diagnóstico	9
1.2.1 Ausencia de Registro de Accesos	9
1.2.2 Inexistencia de Canal de Reporte de Incidentes	9
1.2.3 Gestión Informal de Objetos Perdidos	10
1.2.4 Ausencia de Notificaciones y Alertas	10
1.3 Riesgos Identificados del Problema	10
1.4 Justificación del Proyecto	10
2 Objetivos del Sistema	12
2.1 Objetivo General	12
2.2 Objetivos Específicos	12
2.3 Metas e Indicadores de Éxito	13
3 Alcance del Sistema	14
3.1 Módulos Incluidos	14
3.2 Limitaciones y Exclusiones	14
3.3 Usuarios Objetivo	14
4 Requerimientos del Sistema	16
4.1 Requerimientos Funcionales	16
4.1.1 RF01 — Autenticación y Gestión de Sesiones	16
4.1.2 RF02 — Control de Accesos	17
4.1.3 RF03 — Reporte de Incidentes	17
4.1.4 RF04 — Gestión de Objetos Perdidos	18
4.1.5 RF05 — Notificaciones	18
4.1.6 RF06 — Auditoría y Administración	19
4.2 Requerimientos No Funcionales	19
4.2.1 RNF01 — Rendimiento	19
4.2.2 RNF02 — Seguridad	19
4.2.3 RNF03 — Disponibilidad y Confiabilidad	20
4.2.4 RNF04 — Usabilidad	20
4.2.5 RNF05 — Mantenibilidad y Escalabilidad	20
4.2.6 RNF06 — Compatibilidad	21

5	Roles del Sistema	22
5.1	Descripción General de Roles	22
5.2	Matriz de Permisos por Módulo	23
6	Arquitectura y Stack Tecnológico	24
6.1	Visión General de la Arquitectura	24
6.2	Stack Tecnológico	24
6.2.1	Frontend	25
6.2.2	Backend — Microservicios	25
6.2.3	Base de Datos e Infraestructura	26
6.3	Descripción de Microservicios	26
6.4	Flujo de Autenticación JWT	27
7	Funcionalidades Clave del Sistema	28
7.1	Control de Accesos	28
7.2	Reporte de Incidentes	28
7.3	Gestión de Objetos Perdidos	29
7.4	Sistema de Notificaciones	29
7.5	Auditoría del Sistema	30
8	Modelos de Base de Datos	31
8.1	Estrategia de Base de Datos	31
8.2	Esquemas Principales	31
8.2.1	Colección <code>usuarios</code> — Auth Service	31
8.2.2	Colección <code>registros</code> — Accesos Service	32
8.2.3	Colección <code>incidentes</code> — Incidentes Service	32
8.2.4	Colección <code>auditoria</code> — Auth Service	33
9	Metodología de Desarrollo	34
9.1	Scrum — Marco de Trabajo Ágil	34
9.1.1	¿Por qué Scrum?	34
9.1.2	Ciclo Scrum del Proyecto	34
9.2	Sprints del Proyecto	35
9.3	Herramientas de Gestión	35
10	Estimación de Esfuerzo y Presupuesto	36
10.1	Desglose de Horas por Módulo	36
10.2	Presupuesto del Proyecto	37
10.3	Supuestos de la Estimación	37
11	Roadmap del Proyecto	39
11.1	Plan de Fases	39
12	Gestión de Riesgos	40

12.1 Metodología de Análisis de Riesgos	40
12.2 Registro de Riesgos	40
12.3 Mapa de Calor de Riesgos	42
13 Factores Críticos de Éxito	43
14 Impacto Esperado	45
14.1 Indicadores de Impacto	45
14.2 Métricas de Impacto Proyectadas	46
15 Instalación y Despliegue	47
15.1 Requisitos Previos	47
15.2 Instalación Local — Modo Desarrollo	47
15.3 Tabla de Puertos del Sistema	48
15.4 Despliegue en Producción	49
16 Referencia de la API REST	50
16.1 Convenciones Generales	50
16.2 Códigos de Respuesta HTTP	50
16.3 Endpoints por Servicio	50
16.3.1 Auth Service — /api/v1/auth	51
16.3.2 Accesos Service — /api/v1/accesos	51
16.3.3 Incidentes Service — /api/v1/incidentes	52
16.3.4 Objetos Service — /api/v1/objetos	52
17 Mantenimiento y Operación	53
17.1 Monitoreo del Sistema en Producción	53
17.2 Política de Respaldos	53
17.3 Actualizaciones del Sistema	54
17.4 Renovación del Certificado SSL	54
18 Glosario Técnico	55
19 Conclusiones y Trabajo Futuro	57
19.1 Conclusiones	57
19.2 Trabajo Futuro — Versión 2.0	58

Índice de figuras

5.1	Roles del sistema y sus permisos principales	22
6.1	Arquitectura de microservicios del Sistema de Seguridad Universitaria .	24
6.2	Flujo de autenticación y verificación de token JWT	27
7.1	Flujo de estados de un incidente	29
9.1	Ciclo Scrum aplicado al proyecto	34
10.1	Distribución del presupuesto por rubro	37
11.1	Roadmap completo del proyecto — 13 semanas	39
12.1	Mapa de calor de riesgos del proyecto	42
14.1	Métricas de impacto proyectadas al primer año de operación	46

Índice de cuadros

1.1	Riesgos derivados de la situación actual	10
2.1	Indicadores de éxito del proyecto	13
4.1	Requerimientos de autenticación	16
4.2	Requerimientos de control de accesos	17
4.3	Requerimientos de reporte de incidentes	17
4.4	Requerimientos de objetos perdidos	18
4.5	Requerimientos de notificaciones	18
4.6	Requerimientos de auditoría y administración	19
5.1	Matriz completa de permisos por rol y módulo	23
6.1	Tecnologías del frontend	25
6.2	Tecnologías del backend	25
6.3	Infraestructura y base de datos	26
6.4	Microservicios del sistema con puertos y responsabilidades	26
7.1	Tipos de notificación y destinatarios	29
8.1	Bases de datos por microservicio	31
9.1	Distribución de sprints del proyecto	35
10.1	Estimación de horas por módulo funcional	36
10.2	Desglose del presupuesto en Pesos Colombianos (COP)	37
12.1	Matriz de evaluación de riesgos — referencia	40
12.2	Registro completo de riesgos del proyecto	40
13.1	Factores críticos de éxito y acciones asociadas	44
14.1	Beneficios esperados por actor del sistema	45
15.1	Requisitos de entorno para desarrollo y producción	47
15.2	Puertos utilizados por el sistema	48
16.1	Códigos de respuesta utilizados en la API	50
16.2	Endpoints del Auth Service	51
16.3	Endpoints del Accesos Service	51
16.4	Endpoints del Incidentes Service	52
16.5	Endpoints del Objetos Service	52

17.1 Política de respaldos del sistema	53
19.1 Funcionalidades propuestas para versiones futuras	58

Capítulo 1

Problemática

1.1 Contexto Institucional

Las instituciones universitarias modernas enfrentan el desafío permanente de garantizar la seguridad de su comunidad académica —estudiantes, docentes y personal administrativo— así como de los visitantes externos que acceden cotidianamente a sus instalaciones. La gestión manual y no sistematizada de estos procesos genera vacíos de información, demoras en la respuesta a emergencias y ausencia de trazabilidad en los eventos de seguridad.

La institución objetivo de este proyecto carecía por completo de herramientas digitales de seguridad, dependiendo exclusivamente de la supervisión visual del personal de celadores y registros en papel, métodos que resultan insuficientes en un entorno de alta afluencia de personas.

1.2 Situación Actual — Diagnóstico

1.2.1 Ausencia de Registro de Accesos

No existía ningún mecanismo de identificación digital de las personas que ingresan o salen de la institución. Los celadores operaban únicamente mediante reconocimiento visual, sin dejar evidencia de quién, cuándo y con qué propósito accedió al campus. Esta situación impide cualquier análisis posterior de movimientos y deja la institución expuesta a ingresos no autorizados.

1.2.2 Inexistencia de Canal de Reporte de Incidentes

No existía un mecanismo centralizado para reportar eventos de seguridad como robos, accidentes, situaciones de riesgo o emergencias en el campus. Los incidentes se comunicaban de forma verbal o mediante llamadas telefónicas directas, sin dejar registro, sin seguimiento y sin posibilidad de medir tiempos de respuesta.

1.2.3 Gestión Informal de Objetos Perdidos

Las pertenencias extraviadas por la comunidad universitaria no contaban con ningún sistema de registro. La recuperación de objetos dependía de la suerte o del conocimiento personal entre los afectados, resultando en una tasa de recuperación prácticamente nula.

1.2.4 Ausencia de Notificaciones y Alertas

La administración y el personal de seguridad no recibían alertas automáticas ante incidentes críticos. La cadena de comunicación era lenta, informal y propensa a la pérdida de información, lo que resultaba en respuestas tardías e ineficaces ante emergencias.

1.3 Riesgos Identificados del Problema

La siguiente tabla resume los riesgos concretos derivados de la situación actual:

Cuadro 1.1: Riesgos derivados de la situación actual

Riesgo	Descripción y consecuencias
Robos y hurtos	El ingreso de individuos no verificados incrementa la probabilidad de robos dentro del campus, afectando bienes personales e institucionales.
Emergencias no controladas	Sin monitoreo ni canal de reporte, la respuesta a incidentes críticos es lenta e ineficiente, poniendo en riesgo la integridad física de la comunidad.
Pérdida de objetos personales	Sin sistema de objetos perdidos, recuperar pertenencias extraviadas es prácticamente imposible.
Ingreso de personas sospechosas	Sin verificación de identidad, cualquier persona externa puede acceder libremente a las instalaciones.
Falta de evidencia	Sin registros digitales, no es posible rastrear eventos de seguridad pasados ni presentar evidencia ante situaciones legales.
Decisiones sin datos	La administración carece de estadísticas reales que permitan tomar decisiones estratégicas sobre la seguridad del campus.

1.4 Justificación del Proyecto

La digitalización de los procesos de seguridad universitaria no es una mejora opcional sino una necesidad urgente. La implementación de un sistema integrado permitirá:

- ✓ Eliminar los puntos ciegos en el control de accesos.

- ✓ Establecer un canal formal, trazable y auditable de reporte de incidentes.
- ✓ Crear una base de datos histórica de eventos de seguridad para análisis estratégico.
- ✓ Reducir significativamente el tiempo de respuesta ante emergencias.
- ✓ Proporcionar a la comunidad universitaria herramientas directas de participación en la seguridad.
- ✓ Generar reportes y estadísticas que respalden la toma de decisiones institucionales.

Capítulo 2

Objetivos del Sistema

2.1 Objetivo General

Desarrollar e implementar una **plataforma web integral de seguridad universitaria** que centralice el registro digital de accesos al campus, el reporte y seguimiento de incidentes de seguridad, y la gestión de objetos perdidos o encontrados, garantizando trazabilidad completa, diferenciación de roles y disponibilidad continua del servicio.

2.2 Objetivos Específicos

1. **Registro de accesos:** Implementar un módulo que registre digitalmente las entradas y salidas de personas —usuarios institucionales y visitantes externos— con historial completo, estadísticas diarias y filtros avanzados de consulta.
2. **Canal de incidentes:** Diseñar e implementar un formulario centralizado de reporte de incidentes de seguridad con flujo de estados (Reportado → En Revisión → Resuelto) y notificaciones automáticas al personal competente.
3. **Gestión de objetos perdidos:** Crear un tablón comunitario digital para la publicación, búsqueda y recuperación de objetos perdidos o encontrados, con sistema de comentarios y marcado de resolución.
4. **Notificaciones en tiempo real:** Implementar un servicio de alertas que notifique inmediatamente al personal de seguridad y administración ante nuevos reportes o cambios de estado en incidentes.
5. **Auditoría y reportes:** Generar un registro inmutable de todas las acciones realizadas en el sistema para garantizar trazabilidad, transparencia y soporte en decisiones estratégicas.
6. **Gestión de roles:** Establecer un sistema de acceso diferenciado que defina claramente los permisos y capacidades de cada tipo de usuario: Usuario, Visitante, Celador y Administrador.

7. **Arquitectura escalable:** Construir la plataforma sobre una arquitectura de microservicios independientes que permita escalar, mantener y actualizar cada módulo sin afectar al resto del sistema.
8. **Seguridad de la información:** Implementar autenticación JWT con expiración controlada, hash bcrypt para contraseñas, y control de roles en cada endpoint de la API.
9. **Despliegue en producción:** Configurar un entorno de producción con dominio propio, certificado SSL, proxy inverso Nginx y gestor de procesos PM2 para garantizar disponibilidad 24/7.

2.3 Metas e Indicadores de Éxito

Cuadro 2.1: Indicadores de éxito del proyecto

Módulo	Indicador	Meta	Unidad
Control de accesos	Accesos registrados digitalmente	100	%
Incidentes	Reducción tiempo de respuesta	80	%
Objetos perdidos	Objetos gestionados en sistema	95	%
Disponibilidad	Uptime del sistema en producción	99,5	%
Seguridad	Intentos de acceso no autorizado bloqueados	100	%
Adopción	Usuarios activos en primeros 3 meses	80	%

Capítulo 3

Alcance del Sistema

3.1 Módulos Incluidos

El sistema cubre los siguientes módulos funcionales dentro del alcance del proyecto:

Módulos dentro del alcance

1. **Control de Accesos** — registro de entradas/salidas institucionales y de visitantes.
2. **Reporte de Incidentes** — creación, seguimiento y resolución de reportes de seguridad.
3. **Objetos Perdidos** — tablón comunitario con comentarios y estados de recuperación.
4. **Notificaciones** — alertas en tiempo real al personal de seguridad.
5. **Auditoría** — registro inmutable de acciones del sistema.
6. **Administración** — gestión de usuarios, roles y configuración del sistema.

3.2 Limitaciones y Exclusiones

Fuera del alcance — versión 1.0

- Integración con sistemas biométricos o de reconocimiento facial.
- Aplicación móvil nativa (iOS o Android).
- Integración con cámaras de seguridad o CCTV.
- Módulo de facturación o pagos.
- Integración con sistemas ERP o académicos institucionales preexistentes.
- Soporte para múltiples campus simultáneos (versión 1.0 es monoinstitucional).

3.3 Usuarios Objetivo

El sistema está diseñado para ser utilizado por la comunidad universitaria completa: estudiantes, docentes, personal administrativo, personal de seguridad (celadores) y ad-

ministradores del sistema. Los visitantes externos son registrados en el sistema pero no tienen acceso autónomo a la plataforma.

Capítulo 4

Requerimientos del Sistema

4.1 Requerimientos Funcionales

4.1.1 RF01 — Autenticación y Gestión de Sesiones

Cuadro 4.1: Requerimientos de autenticación

ID	Prioridad	Descripción
RF01.1	Alta	El sistema debe permitir el inicio de sesión mediante número de documento y contraseña.
RF01.2	Alta	El sistema debe generar un token JWT firmado con expiración de 24 horas al autenticar exitosamente.
RF01.3	Alta	El sistema debe rechazar peticiones sin token válido con código HTTP 401.
RF01.4	Alta	El sistema debe verificar el rol del usuario y denegar acceso a rutas no autorizadas con código HTTP 403.
RF01.5	Media	El sistema debe permitir al administrador bloquear cuentas de usuarios activos.
RF01.6	Media	El sistema debe registrar en auditoría cada inicio de sesión exitoso y fallido.

4.1.2 RF02 — Control de Accesos

Cuadro 4.2: Requerimientos de control de accesos

ID	Prioridad	Descripción
RF02.1	Alta	El celador debe poder registrar la entrada o salida de un usuario institucional mediante su número de documento.
RF02.2	Alta	El celador debe poder registrar accesos de visitantes externos con nombre, documento, motivo y persona a visitar.
RF02.3	Alta	El sistema debe almacenar fecha y hora exacta de cada registro de acceso.
RF02.4	Alta	El celador y el administrador deben poder consultar el historial completo de accesos con filtros por fecha, tipo y nombre.
RF02.5	Media	El sistema debe mostrar estadísticas del día actual: total de entradas, salidas e ingresos activos.
RF02.6	Media	El administrador debe poder exportar el historial de accesos con filtros de fecha.
RF02.7	Baja	El sistema debe alertar si un usuario registra múltiples entradas sin salida correspondiente.

4.1.3 RF03 — Reporte de Incidentes

Cuadro 4.3: Requerimientos de reporte de incidentes

ID	Prioridad	Descripción
RF03.1	Alta	Cualquier usuario autenticado (usuario, celador, administrador) debe poder crear un reporte de incidente.
RF03.2	Alta	El reporte debe incluir: tipo, severidad, ubicación, fecha del incidente y descripción (máx. 1000 caracteres).
RF03.3	Alta	El sistema debe soportar los estados: Reportado, En Revisión y Resuelto.
RF03.4	Alta	Solo celadores y administradores pueden cambiar el estado de un incidente.
RF03.5	Alta	Al crearse un incidente, el sistema debe notificar automáticamente a todos los celadores y administradores.
RF03.6	Media	El usuario que creó el reporte debe poder consultar su estado y las observaciones de gestión.
RF03.7	Media	El administrador debe poder editar cualquier reporte y asignar observaciones de resolución.
RF03.8	Baja	El sistema debe soportar al menos cuatro niveles de severidad: Baja, Media, Alta y Crítica.

4.1.4 RF04 — Gestión de Objetos Perdidos

Cuadro 4.4: Requerimientos de objetos perdidos

ID	Prioridad	Descripción
RF04.1	Alta	Cualquier usuario autenticado debe poder publicar un objeto perdido o encontrado.
RF04.2	Alta	La publicación debe incluir: tipo de reporte, nombre del objeto, descripción, ubicación y fecha del evento.
RF04.3	Media	El sistema debe permitir adjuntar una imagen al reporte de objeto perdido.
RF04.4	Alta	Todos los usuarios autenticados deben poder ver el listado de objetos y agregar comentarios.
RF04.5	Alta	El administrador debe poder marcar un objeto como devuelto/encontrado.
RF04.6	Media	El sistema debe notificar al publicador cuando se agregue un comentario a su reporte.
RF04.7	Media	El sistema debe soportar búsqueda y filtrado de objetos por nombre, tipo y estado.

4.1.5 RF05 — Notificaciones

Cuadro 4.5: Requerimientos de notificaciones

ID	Prioridad	Descripción
RF05.1	Alta	El sistema debe crear notificaciones automáticas al registrarse un nuevo incidente.
RF05.2	Alta	El sistema debe crear notificaciones al cambiar el estado de un incidente.
RF05.3	Media	El sistema debe crear notificaciones al publicarse un nuevo objeto perdido.
RF05.4	Media	El sistema debe crear notificaciones al recibir un comentario en un objeto perdido.
RF05.5	Alta	Cada usuario debe poder consultar sus propias notificaciones no leídas.
RF05.6	Media	El usuario debe poder marcar notificaciones individuales o todas como leídas.

4.1.6 RF06 — Auditoría y Administración

Cuadro 4.6: Requerimientos de auditoría y administración

ID	Prioridad	Descripción
RF06.1	Alta	El sistema debe registrar automáticamente cada acción relevante: usuario, acción, módulo, IP y fecha.
RF06.2	Alta	Solo el administrador puede consultar el registro completo de auditoría.
RF06.3	Alta	El administrador debe poder crear, editar, bloquear y eliminar usuarios del sistema.
RF06.4	Alta	El administrador debe poder configurar los tipos de incidente disponibles en el sistema.
RF06.5	Media	El administrador debe poder generar estadísticas de accesos por rango de fecha.
RF06.6	Media	El registro de auditoría debe ser inmutable (no editable ni eliminable por ningún usuario).

4.2 Requerimientos No Funcionales

4.2.1 RNF01 — Rendimiento

- **RNF01.1** — El tiempo de respuesta de cualquier endpoint de la API no debe superar los **2 segundos** bajo condiciones normales de carga.
- **RNF01.2** — El frontend debe cargarse completamente en menos de **3 segundos** en conexiones de banda ancha estándar.
- **RNF01.3** — El sistema debe soportar al menos **100 usuarios concurrentes** sin degradación perceptible del servicio.
- **RNF01.4** — Las consultas al historial de accesos deben devolver resultados en menos de **1 segundo** para rangos de hasta 30 días.

4.2.2 RNF02 — Seguridad

- **RNF02.1** — Todas las contraseñas deben almacenarse como hash bcrypt con factor de coste mínimo de 10.
- **RNF02.2** — Los tokens JWT deben tener una expiración máxima de 24 horas.
- **RNF02.3** — Toda la comunicación en producción debe realizarse sobre HTTPS con certificado SSL válido.

- **RNF02.4** — Los puertos de los microservicios (3001–3005) deben estar cerrados al exterior; solo Nginx (80/443) expuesto.
- **RNF02.5** — Las entradas de usuario deben sanitizarse para prevenir inyecciones NoSQL y XSS.
- **RNF02.6** — Las cabeceras de seguridad HTTP (HSTS, X-Frame-Options, X-Content-Type-Options) deben estar configuradas en Nginx.

4.2.3 RNF03 — Disponibilidad y Confiabilidad

- **RNF03.1** — El sistema debe tener una disponibilidad del **99,5%** (uptime), equivalente a menos de 44 horas de inactividad al año.
- **RNF03.2** — PM2 debe reiniciar automáticamente cualquier microservicio que falle.
- **RNF03.3** — Un fallo en un microservicio no debe afectar la operación de los demás.
- **RNF03.4** — MongoDB Atlas debe tener backups automáticos habilitados con punto de recuperación diario.

4.2.4 RNF04 — Usabilidad

- **RNF04.1** — La interfaz debe ser responsiva y funcionar correctamente en dispositivos de escritorio y tablets.
- **RNF04.2** — Los celadores deben poder registrar un acceso en menos de **30 segundos** tras recibir capacitación básica.
- **RNF04.3** — El sistema debe mostrar mensajes de error comprensibles y orientados al usuario, no mensajes técnicos.
- **RNF04.4** — Debe existir retroalimentación visual inmediata ante cualquier acción del usuario (estados de carga, confirmaciones, errores).

4.2.5 RNF05 — Mantenibilidad y Escalabilidad

- **RNF05.1** — Cada microservicio debe poder escalarse de forma independiente sin modificar los demás.
- **RNF05.2** — El código debe seguir una estructura de carpetas convencional y documentada en cada microservicio.

- **RNF05.3** — Las variables de configuración sensibles deben gestionarse mediante archivos `.env` excluidos del repositorio.
- **RNF05.4** — Cada microservicio debe incluir al menos las pruebas de integración de sus endpoints principales.

4.2.6 RNF06 — Compatibilidad

- **RNF06.1** — El frontend debe ser compatible con los navegadores: Chrome 110+, Firefox 110+, Safari 16+ y Edge 110+.
- **RNF06.2** — La API debe seguir las convenciones REST y devolver siempre JSON con cabecera `Content-Type: application/json`.
- **RNF06.3** — Las fechas deben transmitirse y almacenarse en formato ISO 8601 UTC.

Capítulo 5

Roles del Sistema

5.1 Descripción General de Roles

El sistema implementa un modelo de control de acceso basado en roles (RBAC — *Role-Based Access Control*). Cada usuario tiene asignado exactamente un rol que determina los módulos y acciones a los que puede acceder.

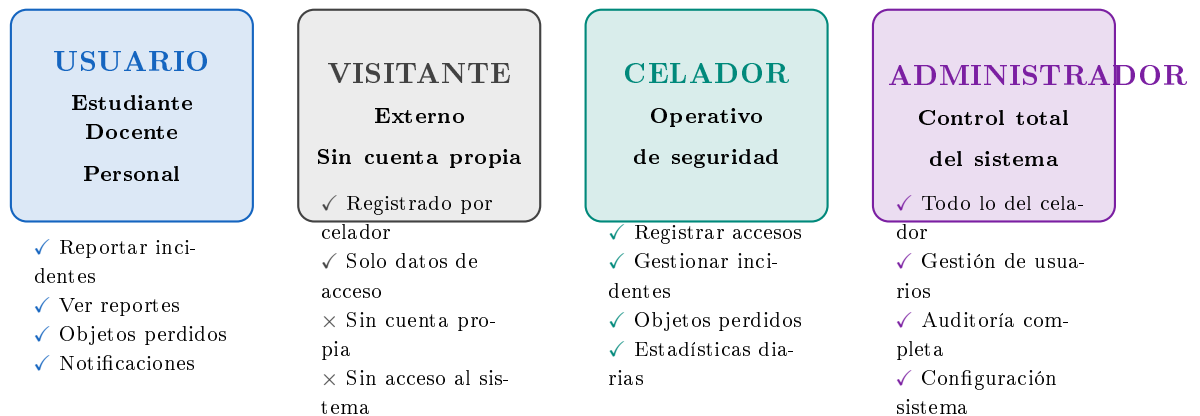


Figura 5.1: Roles del sistema y sus permisos principales

5.2 Matriz de Permisos por Módulo

Cuadro 5.1: Matriz completa de permisos por rol y módulo

Acción	Usuario	Visitante	Celador	Administrador
<i>Autenticación</i>				
Iniciar sesión	✓	×	✓	✓
<i>Control de Accesos</i>				
Registrar acceso institucional	×	×	✓	✓
Registrar acceso visitante	×	×	✓	✓
Ver historial completo	×	×	✓	✓
Ver registros del día	×	×	✓	✓
Ver estadísticas avanzadas	×	×	×	✓
<i>Incidentes</i>				
Crear reporte	✓	×	✓	✓
Ver mis reportes	✓	×	✓	✓
Ver todos los reportes	×	×	✓	✓
Tomar en revisión	×	×	✓	✓
Marcar como resuelto	×	×	✓	✓
Editar cualquier reporte	×	×	×	✓
<i>Objetos Perdidos</i>				
Publicar objeto	✓	×	✓	✓
Ver listado	✓	×	✓	✓
Agregar comentario	✓	×	✓	✓
Marcar como devuelto	×	×	×	✓
Eliminar publicación	×	×	×	✓
<i>Administración</i>				
Crear/editar usuarios	×	×	×	✓
Bloquear/activar usuarios	×	×	×	✓
Eliminar usuarios	×	×	×	✓
Ver auditoría	×	×	×	✓
Configurar tipos de incidente	×	×	×	✓
<i>Notificaciones</i>				
Ver mis notificaciones	✓	×	✓	✓
Marcar como leídas	✓	×	✓	✓

Capítulo 6

Arquitectura y Stack Tecnológico

6.1 Visión General de la Arquitectura

La plataforma implementa una **arquitectura de microservicios**: cada módulo funcional corre como un proceso Node.js independiente, expone su propia API REST y posee su propia base de datos MongoDB. Esto garantiza aislamiento total, escalabilidad independiente y resiliencia ante fallos.

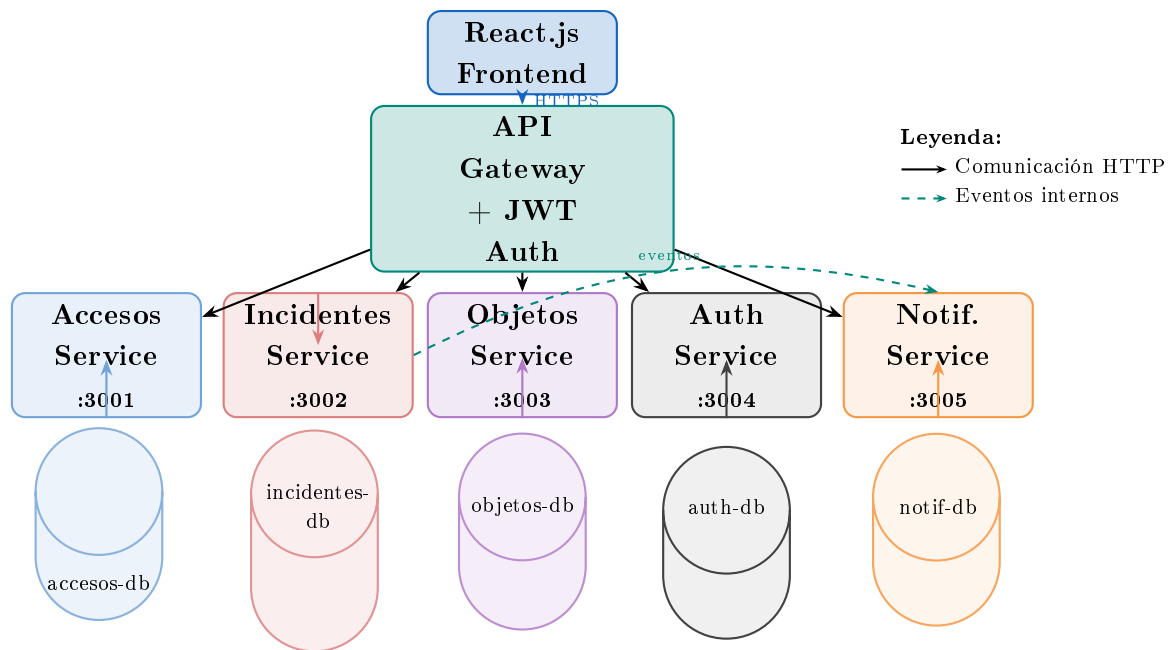


Figura 6.1: Arquitectura de microservicios del Sistema de Seguridad Universitaria

6.2 Stack Tecnológico

6.2.1 Frontend

Cuadro 6.1: Tecnologías del frontend

Tecnología	Versión	Propósito
React.js	18.x	Framework de interfaz de usuario reactiva con componentes reutilizables.
Tailwind CSS	3.x	Framework utilitario para diseño moderno y responsivo sin CSS personalizado.
React Router	6.x	Gestión de navegación y rutas protegidas entre vistas de la SPA.
Axios	1.x	Cliente HTTP para consumo de los microservicios REST con interceptores de token.
React Context API	—	Gestión del estado global de autenticación y datos del usuario.

6.2.2 Backend — Microservicios

Cuadro 6.2: Tecnologías del backend

Tecnología	Versión	Propósito
Node.js	18 LTS	Entorno de ejecución JavaScript para los microservicios.
Express.js	4.x	Framework minimalista para definición de rutas y middlewares REST.
JWT (jsonwebtoken)	9.x	Generación y verificación de tokens de autenticación firmados.
bcrypt	5.x	Hashing seguro de contraseñas con factor de coste configurable.
Mongoose ODM	7.x	Modelado de esquemas, validación y consultas sobre MongoDB.
cors	2.x	Configuración de políticas de origen cruzado para el frontend.
dotenv	16.x	Carga de variables de entorno desde archivos <code>.env</code> .

6.2.3 Base de Datos e Infraestructura

Cuadro 6.3: Infraestructura y base de datos

Tecnología	Versión	Propósito
MongoDB	6.x	Base de datos NoSQL orientada a documentos. Una BD por microservicio.
MongoDB Atlas	Cloud	Servicio gestionado con backups automáticos y alta disponibilidad.
Nginx	1.24	Proxy inverso, terminación SSL y servidor de archivos estáticos del frontend.
PM2	5.x	Gestor de procesos Node.js en producción con reinicio automático.
Let's Encrypt	—	Autoridad certificadora gratuita para certificados SSL/TLS de 90 días.
Certbot	—	Cliente de Let's Encrypt para emisión y renovación automática de certificados.
Ubuntu Server	22.04 LTS	Sistema operativo del servidor VPS de producción.
Git + GitHub	—	Control de versiones y repositorio del código fuente.

6.3 Descripción de Microservicios

Cuadro 6.4: Microservicios del sistema con puertos y responsabilidades

Servicio	Puerto	Responsabilidad
Auth Service	3004	Autenticación, gestión de usuarios, generación de JWT, auditoría y configuración del sistema.
Accesos Service	3001	Registro y consulta de movimientos de entrada/-salida de institucionales y visitantes.
Incidentes Service	3002	Ciclo de vida completo de reportes de seguridad: creación, estados, observaciones y resolución.
Objetos Service	3003	Tablón de objetos perdidos/encontrados con comentarios, imágenes y estados de devolución.
Notificaciones Service	3005	Recepción de eventos internos y exposición de notificaciones persistentes por usuario.

6.4 Flujo de Autenticación JWT

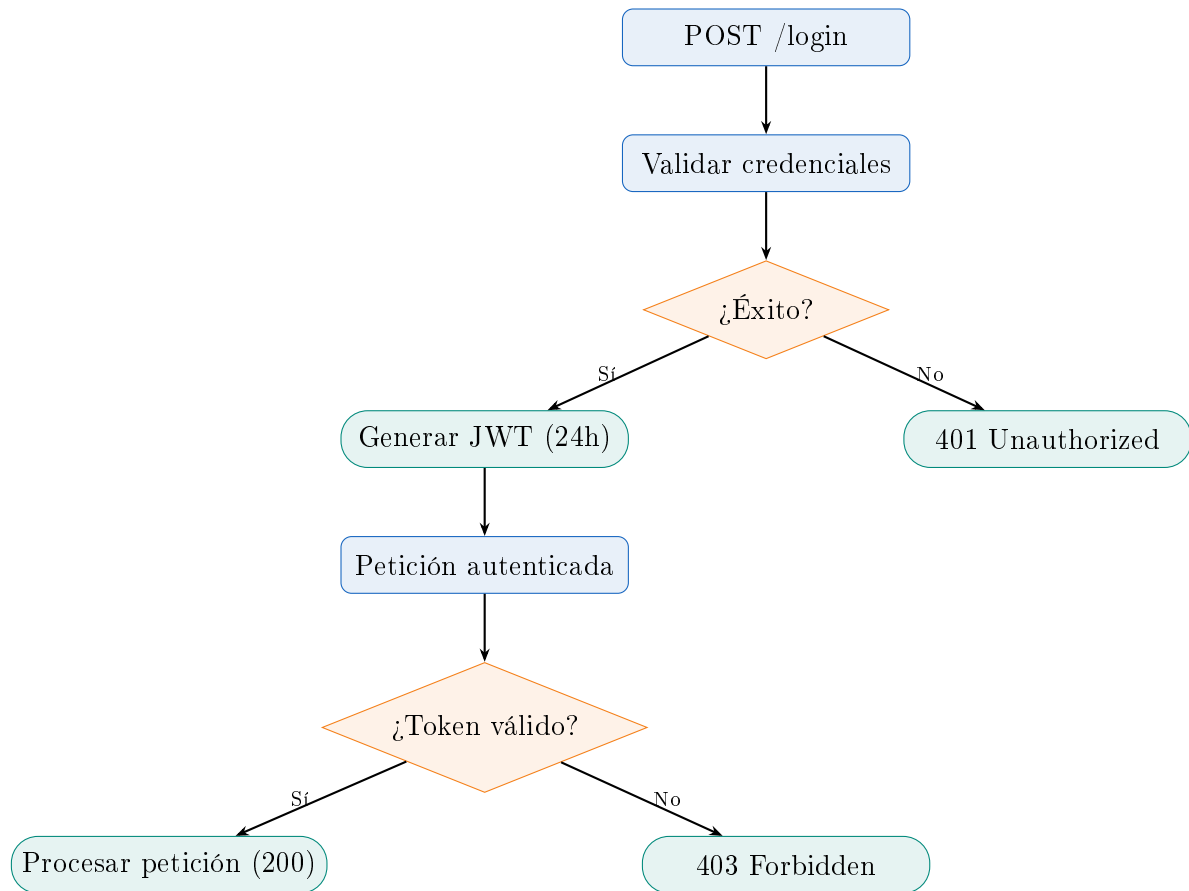


Figura 6.2: Flujo de autenticación y verificación de token JWT

Capítulo 7

Funcionalidades Clave del Sistema

7.1 Control de Accesos

El módulo de Control de Accesos permite al personal de seguridad registrar digitalmente todos los movimientos de entrada y salida del campus en tiempo real.

Características principales:

- Registro de accesos institucionales mediante número de documento, con validación automática contra la base de usuarios.
- Registro de visitantes externos con captura de datos: nombre, documento, motivo de visita y persona a visitar.
- Historial completo filtrable por fecha, tipo de persona y nombre.
- Dashboard diario con conteo de entradas, salidas y personas activas en el campus.
- Estadísticas por rango de fecha para análisis de afluencia (solo administrador).

Flujo de registro de acceso institucional:

1. El celador ingresa el número de documento del usuario.
2. El sistema consulta al Auth Service para verificar que el usuario existe y está activo.
3. Se registra el movimiento (Entrada/Salida) con fecha y hora exacta.
4. El registro queda almacenado en `accesos-db` con referencia al celador que lo realizó.
5. La respuesta confirma el registro con los datos del usuario y el movimiento.

7.2 Reporte de Incidentes

El módulo de Incidentes centraliza todos los eventos de seguridad reportados en el campus con un flujo de gestión completo.

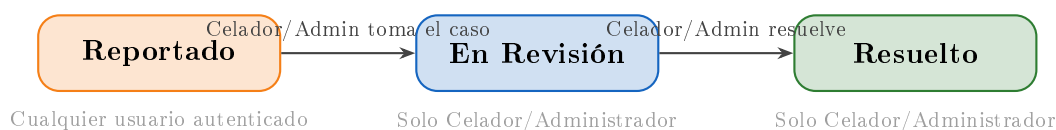
Estados del flujo de incidentes:

Figura 7.1: Flujo de estados de un incidente

7.3 Gestión de Objetos Perdidos

El módulo de Objetos Perdidos funciona como un tablón comunitario digital donde cualquier miembro de la universidad puede publicar objetos perdidos o encontrados.

Características:

- Dos tipos de publicación: *Perdido* (busco mi objeto) y *Encontrado* (encontré un objeto ajeno).
- Soporte para imagen adjunta (URL o base64).
- Sistema de comentarios anidados para facilitar la comunicación entre partes.
- Filtros de búsqueda por nombre, tipo y estado (Activo/Devuelto).
- Notificación automática al publicador cuando se agrega un comentario.
- El administrador puede marcar el objeto como devuelto cerrando el caso.

7.4 Sistema de Notificaciones

El servicio de notificaciones actúa como un bus de eventos que conecta los demás módulos y mantiene informado al personal de seguridad en tiempo real.

Cuadro 7.1: Tipos de notificación y destinatarios

Tipo	Destinatario	Disparador
NuevoIncidente	Celadores y Administradores	Se crea un nuevo reporte de incidente.
CambioEstado	Usuario reportador	El estado del incidente cambia.
ObjetoPerdido	Celadores y Administradores	Se publica un nuevo objeto perdido.
NuevoComentario	Publicador del objeto	Alguien comenta en su publicación.

7.5 Auditoría del Sistema

Cada acción relevante en el sistema queda registrada automáticamente en la colección de auditoría con los siguientes datos: usuario que realizó la acción, nombre del usuario, descripción de la acción, módulo afectado, dirección IP y fecha/hora exacta.

El registro de auditoría es **immutable**: no puede ser editado ni eliminado por ningún usuario, incluyendo el administrador. Solo puede ser consultado.

Capítulo 8

Modelos de Base de Datos

8.1 Estrategia de Base de Datos

El sistema adopta el principio de **una base de datos por microservicio**, garantizando aislamiento total de datos, independencia de despliegue y resiliencia ante fallos parciales. Todas las bases de datos utilizan MongoDB como motor NoSQL orientado a documentos.

Cuadro 8.1: Bases de datos por microservicio

Servicio	Base de datos	Colecciones principales	Entorno
Auth Service	auth-db	usuarios, auditoria, configuracion	MongoDB Atlas
Accesos Service	accesos-db	registros	MongoDB Atlas
Incidentes Service	incidentes-db	incidentes	MongoDB Atlas
Objetos Service	objetos-db	objetos	MongoDB Atlas
Notif. Service	notif-db	notificaciones	MongoDB Atlas

8.2 Esquemas Principales

8.2.1 Colección usuarios — Auth Service

```
1 const usuarioSchema = new mongoose.Schema({
2   nombres:      { type: String, required: true, trim: true },
3   apellidos:    { type: String, required: true, trim: true },
4   numeroDocumento: { type: String, required: true, unique: true
5     },
6   tipoDocumento: { type: String, enum:
7     [ 'CC', 'TI', 'CE', 'Pasaporte' ], default: 'CC' },
8   correo:       { type: String, required: true, unique:
9     true, lowercase: true },
10  telefono:      { type: String },
11  password:      { type: String, required: true }, // Hash
12    bcrypt
13  rol: {
```

```

10     type: String,
11     enum: ['Estudiante', 'Docente', 'Personal
12           Administrativo', 'Celador', 'Administrador'],
13     required: true
14 },
15 estado: { type: String, enum:
16           ['Activo', 'Bloqueado'], default: 'Activo' },
17 }, { timestamps: true });

```

Listing 8.1: Esquema Mongoose del modelo Usuario

8.2.2 Colección registros — Accesos Service

```

1 const registroAccesoSchema = new mongoose.Schema({
2   tipo: { type: String, enum:
3         ['Institucional', 'Visitante'], required: true },
4   movimiento: { type: String, enum: ['Entrada', 'Salida'],
5         required: true },
6   usuarioId: { type: String }, // _id del Auth
7         Service
8   numeroDocumento: { type: String },
9   nombreCompleto: { type: String },
10  visitante: { // Solo para
11        visitantes externos
12    documento: { type: String },
13    nombres: { type: String },
14    apellidos: { type: String },
15    motivoVisita: { type: String },
16    personaVisitar: { type: String },
17  },
18  observaciones: { type: String },
19  registradoPor: { type: String, required: true }, // _id del
20        celador
21  fecha: { type: Date, default: Date.now },
22 }, { timestamps: true });

```

Listing 8.2: Esquema Mongoose del modelo RegistroAcceso

8.2.3 Colección incidentes — Incidentes Service

```

1 const incidenteSchema = new mongoose.Schema({
2   tipo: { type: String, required: true },
3   severidad: { type: String, enum:
4         ['Baja', 'Media', 'Alta', 'Critica'], required: true },
5   ubicacion: { type: String, required: true },

```

```
5  fechaIncidente:    { type: Date, required: true },
6  descripcion:      { type: String, required: true, maxlength:
    1000 },
7  estado:           { type: String, enum: ['Reportado', 'En
    Revision', 'Resuelto'],
8                      default: 'Reportado' },
9  reportadoPor:      { usuarioId: String, nombre: String,
    documento: String, rol: String },
10 gestionadoPor:     { usuarioId: String, nombre: String },
11 observacionesGestion: { type: String },
12 fechaResolucion:   { type: Date },
13 }, { timestamps: true });
```

Listing 8.3: Esquema Mongoose del modelo Incidente

8.2.4 Colección auditoria — Auth Service

```
1  const auditoriaSchema = new mongoose.Schema({
2    usuarioId:      { type: String, required: true },
3    nombreUsuario: { type: String },
4    accion:         { type: String, required: true },
5    modulo: {
6      type: String,
7      enum:
8        ['Accesos', 'Incidentes', 'Objetos', 'Usuarios', 'Configuracion', 'Auth'],
9      required: true
10   },
11   descripcion: { type: String },
12   ip:          { type: String },
13   fecha:       { type: Date, default: Date.now },
14 }, { timestamps: false });
```

Listing 8.4: Esquema del registro de auditoría (inmutable)

Capítulo 9

Metodología de Desarrollo

9.1 Scrum — Marco de Trabajo Ágil

El proyecto fue desarrollado utilizando **Scrum**, un marco de trabajo ágil que permite entregas incrementales y continuas de software funcional, especialmente adecuado cuando los requisitos pueden evolucionar durante el desarrollo.

9.1.1 ¿Por qué Scrum?

Justificación de la elección de Scrum

- **Entregas incrementales:** Cada sprint produce software funcional que puede ser revisado y validado.
- **Adaptabilidad:** Los requisitos pueden ajustarse entre sprints sin desestabilizar el proyecto.
- **Retroalimentación temprana:** Las revisiones frecuentes permiten detectar problemas antes de que se acumulen.
- **Transparencia:** El Product Backlog y las sprint reviews mantienen a todos los stakeholders informados.
- **Equipo pequeño:** Scrum funciona especialmente bien con equipos de 3–9 personas, como el equipo de 4 desarrolladores de este proyecto.

9.1.2 Ciclo Scrum del Proyecto

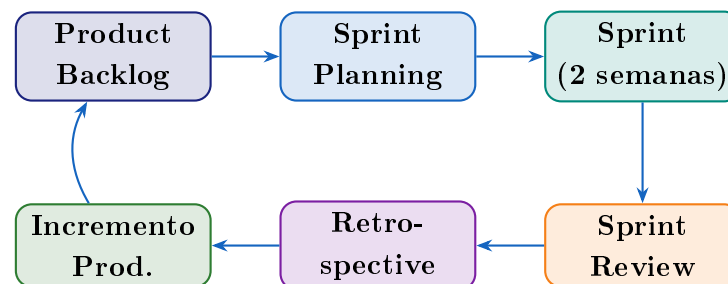


Figura 9.1: Ciclo Scrum aplicado al proyecto

9.2 Sprints del Proyecto

Cuadro 9.1: Distribución de sprints del proyecto

Sprint	Nombre	Funcionalidades cubiertas	Semanas	Horas
S1	Auth & Roles	Autenticación JWT, gestión de usuarios, control de roles, registro en auditoría.	3–4	24
S2	Control Accesos	Registro de accesos institucionales y visitantes, historial, estadísticas diarias.	5–6	40
S3	Incidentes	Creación de reportes, flujo de estados, notificaciones, gestión por celador/admin.	7–8	36
S4	Objetos + Notif.	Tablón de objetos perdidos, comentarios, imágenes, servicio de notificaciones.	9–10	30
S5	Auditoría + Deploy	Módulo de auditoría, configuración del sistema, pruebas E2E, despliegue en producción.	11–13	80
			TOTAL	210 h

Nota: Las 80 horas del Sprint 5 incluyen las 28 h de auditoría/configuración, 30 h de diseño UI/UX transversal y 22 h de pruebas y despliegue.

9.3 Herramientas de Gestión

- **Control de versiones:** Git + GitHub, con branches por feature y pull requests para revisión de código.
- **Gestión de backlog:** Tablero Kanban con columnas: Por hacer / En progreso / En revisión / Hecho.
- **Pruebas de API:** Postman con colecciones organizadas por microservicio y variables de entorno.
- **Documentación colaborativa:** README por microservicio y documentación técnica centralizada.
- **Comunicación del equipo:** Reuniones diarias de 15 minutos (Daily Scrum) y sprint reviews semanales.

Capítulo 10

Estimación de Esfuerzo y Presupuesto

10.1 Desglose de Horas por Módulo

Cuadro 10.1: Estimación de horas por módulo funcional

Módulo	Horas estimadas	Sprint	% del total
Autenticación y roles	24 h	S1	11,4%
Control de accesos	40 h	S2	19,0%
Módulo de incidentes	36 h	S3	17,1%
Objetos perdidos + Notificaciones	30 h	S4	14,3%
Auditoría y configuración	28 h	S5	13,3%
Diseño UI/UX y microservicios	30 h	S1–S5	14,3%
Pruebas, QA y despliegue	22 h	S5	10,5%
TOTAL	210 h	5 Spr.	100%

10.2 Presupuesto del Proyecto

Cuadro 10.2: Desglose del presupuesto en Pesos Colombianos (COP)

Rubro	Detalle	Costo (COP)
Recursos Humanos	4 desarrolladores $\times$ 52,5 h $\times$ \$50.000/h	\$10.500.000
Infraestructura	Servidor VPS, dominio, MongoDB Atlas (M0)	\$2.400.000
Licencias y herramientas	Software de diseño, herramientas de testing, IDEs	\$900.000
Contingencia (10%)	Reserva para imprevistos y ajustes	\$1.380.000
TOTAL ESTIMADO		\$15.180.000 COP

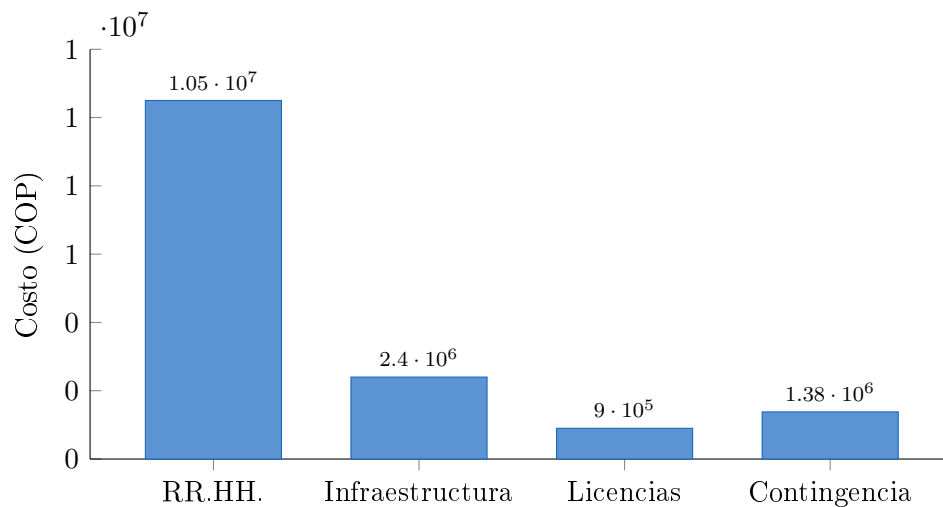


Figura 10.1: Distribución del presupuesto por rubro

10.3 Supuestos de la Estimación

- La tarifa de \$50.000 COP/hora corresponde a desarrolladores júnior-semi-senior con experiencia en el stack MERN.
- Las 52,5 horas por desarrollador se distribuyen a lo largo de los 5 sprints (210 h / 4 devs).

- La infraestructura se cotizó en MongoDB Atlas plan M0 (gratuito) más VPS básico y dominio .com por un año.
- La contingencia del 10% cubre cambios de requisitos menores, horas extra y ajustes de despliegue.
- No se incluyen costos de capacitación a usuarios finales ni mantenimiento post-entrega en esta estimación.

Capítulo 11

Roadmap del Proyecto

11.1 Plan de Fases

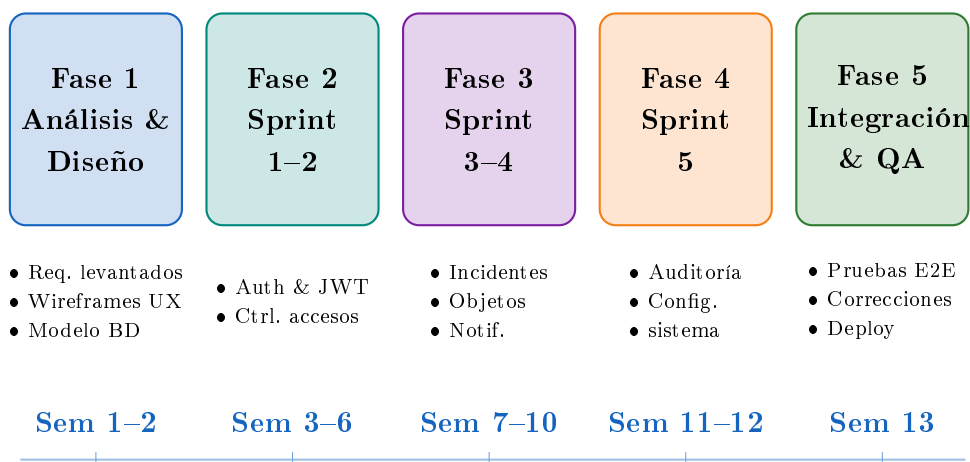


Figura 11.1: Roadmap completo del proyecto — 13 semanas

Resumen del cronograma			
Duración total: 13 semanas	Equipo: 4 desarrolladores	Sprints: 5	Horas totales: 210 horas de trabajo

Capítulo 12

Gestión de Riesgos

12.1 Metodología de Análisis de Riesgos

Los riesgos del proyecto fueron identificados y analizados mediante la metodología estándar de gestión de proyectos de software, evaluando cada riesgo en dos dimensiones: **probabilidad** de ocurrencia (Baja / Media / Alta) e **impacto** potencial sobre el proyecto (Bajo / Medio / Alto).

La **exposición al riesgo** se calcula como el producto de ambas dimensiones y determina la prioridad de atención:

Cuadro 12.1: Matriz de evaluación de riesgos — referencia

	Impacto Bajo	Impacto Medio	Impacto Alto
Prob. Alta	Medio	Alto	Crítico
Prob. Media	Bajo	Medio	Alto
Prob. Baja	Bajo	Bajo	Medio

12.2 Registro de Riesgos

Cuadro 12.2: Registro completo de riesgos del proyecto

Riesgo	Prob.	Impacto	Plan de mitigación
Cambios frecuentes de requisitos	Alta	Alto	Reuniones semanales con stakeholders. Backlog flexible con historias priorizadas. Gestión de cambios documentada.
Disponibilidad del equipo	Media	Alto	Responsabilidades claramente definidas por desarrollador. Buffer de tiempo de 10% en cada sprint. Documentación de trabajo en progreso.

(continuación)

Riesgo	Prob.	Impacto	Plan de mitigación
Integración frontend / microservicios	Media	Medio	Tests de integración continuos desde el Sprint 2. Revisiones de código obligatorias por PR. Contratos de API documentados antes del desarrollo.
Vulnerabilidades de seguridad JWT	Baja	Alto	Tokens con expiración de 24h. Sanitización de todas las entradas. Logs de auditoría completos. Revisión de dependencias con npm audit.
Adopción baja por usuarios	Media	Alto	Capacitaciones presenciales al personal de seguridad. Manual de usuario con capturas de pantalla. Diseño UX intuitivo validado con usuarios reales.
Pérdida de datos en producción	Baja	Alto	Backups automáticos configurados en MongoDB Atlas. Política de recuperación documentada. Pruebas periódicas de restauración.
Fallo del servidor de producción	Baja	Alto	PM2 con reinicio automático. Monitoreo activo con PM2 monit. Procedimiento de respaldo en VPS alternativo documentado.
Deuda técnica acumulada	Media	Medio	Code reviews obligatorios. Refactoring planificado en cada sprint. Estándares de código documentados y revisados.
Dependencias desactualizadas	Baja	Medio	Revisión mensual de versiones de dependencias. Uso de npm audit en cada sprint. Pinning de versiones en package.json.
Vencimiento del certificado SSL	Baja	Alto	Certbot con renovación automática. Alerta de vencimiento en el panel de administración. Renovación manual documentada como backup.

12.3 Mapa de Calor de Riesgos

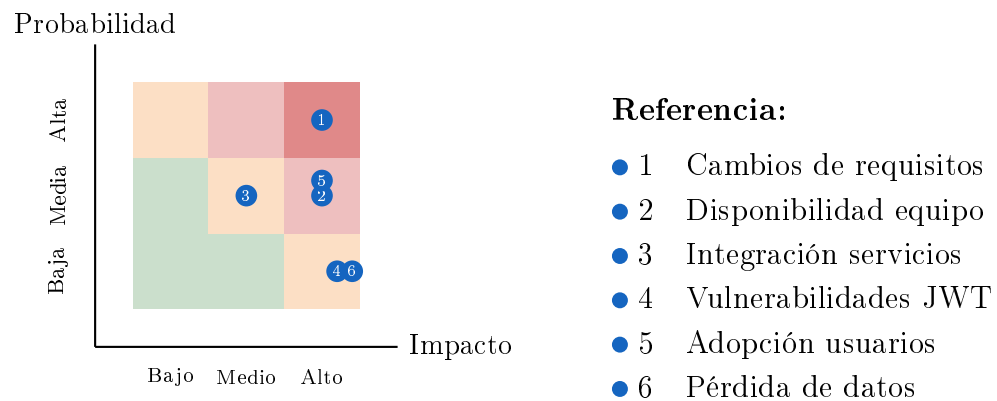


Figura 12.1: Mapa de calor de riesgos del proyecto

Capítulo 13

Factores Críticos de Éxito

Los factores críticos de éxito (FCE) son las condiciones indispensables que deben cumplirse para que el proyecto sea considerado exitoso por todos los stakeholders.

Cuadro 13.1: Factores críticos de éxito y acciones asociadas

Factor	Descripción	Acciones clave
Compromiso institucional	El apoyo de directivos y administración es esencial para la adopción masiva del sistema.	Presentaciones a directivos, demostración de valor temprana, alineación con objetivos institucionales.
Seguridad de la información	Protección robusta con JWT, control de roles y auditoría de cada acción.	Implementar todos los RNF de seguridad, pruebas de penetración básicas, revisión de código enfocada en seguridad.
Facilidad de uso (UX)	Interfaz intuitiva que minimice la curva de aprendizaje, especialmente para celadores.	Prototipos validados con usuarios reales, onboarding guiado, mensajes de error claros.
Alta disponibilidad	El sistema debe operar 24/7, incluso en horario nocturno cuando los celadores trabajan.	PM2 con autorestart, MongoDB Atlas con alta disponibilidad, certificado SSL con renovación automática.
Capacitación continua	Formación adecuada garantiza el uso correcto y reduce errores operativos del personal.	Manual de usuario con capturas, sesiones de capacitación presencial, soporte técnico en las primeras semanas.
Mantenimiento y evolución	Plan de actualizaciones basado en retroalimentación real de usuarios del campus.	Versionado semántico, backlog de mejoras, revisiones trimestrales de funcionalidades.

Capítulo 14

Impacto Esperado

14.1 Indicadores de Impacto

Cuadro 14.1: Beneficios esperados por actor del sistema

Actor	Beneficio principal	Mejora clave
Estudiantes y Docentes	Canal directo y formal de reporte de incidentes de seguridad desde cualquier dispositivo.	Seguridad y respuesta inmediata; participación activa en la seguridad del campus.
Personal administrativo	Módulo de objetos perdidos que facilita la recuperación de pertenencias extraviadas.	Reducción del tiempo de recuperación de un objeto de semanas a horas.
Celadores	Registro digital completo de todos los accesos, eliminando planillas de papel.	Elimina errores humanos, omisiones y facilita la búsqueda retroactiva.
Administración	Visibilidad total del campus en tiempo real con estadísticas y reportes históricos.	Decisiones estratégicas de seguridad basadas en datos reales, no percepciones.
Universidad (institución)	Reducción significativa de incidentes e imagen institucional mejorada.	Campus seguro, trazable y con capacidad de respuesta ante cualquier evento.

14.2 Métricas de Impacto Proyectadas

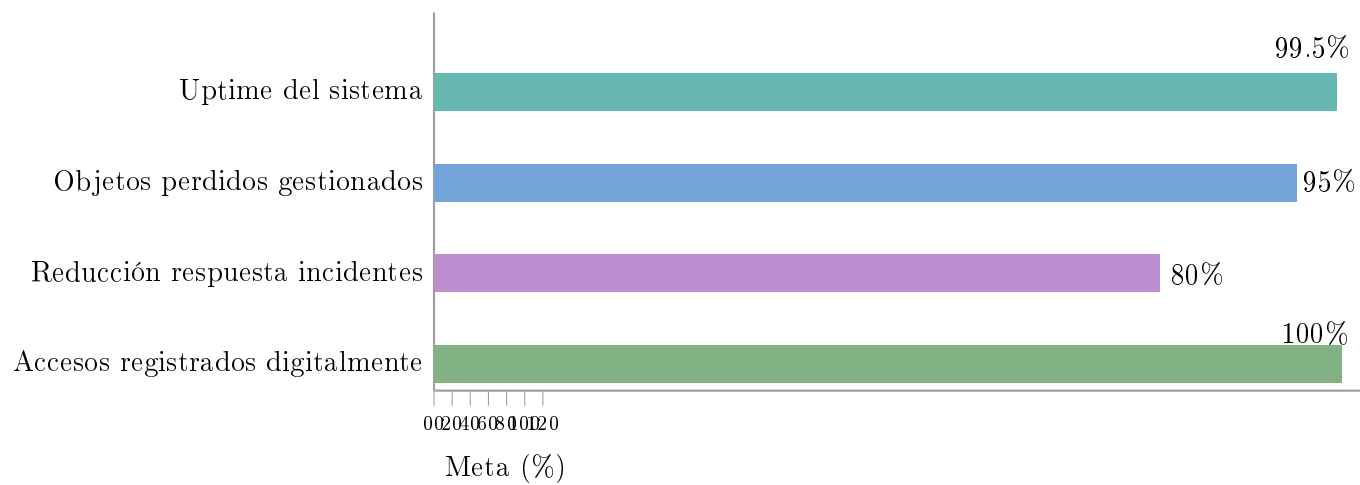


Figura 14.1: Métricas de impacto proyectadas al primer año de operación

Capítulo 15

Instalación y Despliegue

15.1 Requisitos Previos

Cuadro 15.1: Requisitos de entorno para desarrollo y producción

Herramienta	Versión mínima	Propósito	Entorno
Node.js	18 LTS	Runtime de los microservicios	Dev + Prod
npm	9.x	Gestión de dependencias	Dev + Prod
MongoDB	6.x	Base de datos NoSQL	Dev local
MongoDB Atlas	Cloud	BD gestionada en la nube	Producción
Git	2.x	Control de versiones	Dev
Nginx	1.24	Proxy inverso y SSL	Producción
PM2	5.x	Gestor de procesos Node.js	Producción
Certbot	Latest	Certificados SSL Let's Encrypt	Producción
Ubuntu Server	22.04 LTS	SO del servidor VPS	Producción

15.2 Instalación Local — Modo Desarrollo

```
1 # Clonar el repositorio
2 git clone https://github.com/usuario/seguridad-universitaria.git
3 cd seguridad-universitaria
4
5 # Instalar dependencias del frontend
6 cd frontend && npm install && cd ..
7
8 # Instalar dependencias de cada microservicio
9 for svc in auth-service accesos-service incidentes-service \
10     objetos-service notif-service; do
11     cd services/$svc && npm install && cd ../..
12 done
```

Listing 15.1: Clonar el repositorio e instalar dependencias

```

1 # Terminal 1 - Auth Service (Puerto 3004)
2 cd services/auth-service && npm run dev
3
4 # Terminal 2 - Accesos Service (Puerto 3001)
5 cd services/accesos-service && npm run dev
6
7 # Terminal 3 - Incidentes Service (Puerto 3002)
8 cd services/incidentes-service && npm run dev
9
10 # Terminal 4 - Objetos Service (Puerto 3003)
11 cd services/objetos-service && npm run dev
12
13 # Terminal 5 - Notificaciones Service (Puerto 3005)
14 cd services/notif-service && npm run dev
15
16 # Terminal 6 - Frontend React (Puerto 3000)
17 cd frontend && npm start

```

Listing 15.2: Levantar todos los servicios en modo desarrollo

15.3 Tabla de Puertos del Sistema

Cuadro 15.2: Puertos utilizados por el sistema

Servicio	Puerto	Descripción	Exposición
Nginx (HTTP)	80	Redirige todo el tráfico a HTTPS	Exterior
Nginx (HTTPS)	443	Entrada pública del sistema	Exterior
Auth Service	3004	Autenticación, usuarios, auditoría	Solo localhost
Accesos Service	3001	Registro de entradas y salidas	Solo localhost
Incidentes Service	3002	Gestión de reportes	Solo localhost
Objetos Service	3003	Tablón de objetos perdidos	Solo localhost
Notif. Service	3005	Notificaciones y alertas	Solo localhost
MongoDB (dev)	27017	Solo entorno de desarrollo local	Localhost

Importante — Seguridad de puertos

En producción, únicamente los puertos **80 (HTTP)** y **443 (HTTPS)** deben estar expuestos al exterior. Los puertos 3001–3005 deben estar bloqueados por el firewall UFW y solo accesibles desde localhost. Nginx es el único punto de entrada al sistema.

15.4 Despliegue en Producción

La infraestructura de producción está compuesta por un servidor VPS Ubuntu 22.04 LTS, Nginx como proxy inverso, PM2 como gestor de procesos, y MongoDB Atlas como base de datos gestionada en la nube.

```
1 # Actualizar el sistema operativo
2 sudo apt update && sudo apt upgrade -y
3
4 # Instalar Node.js 18 LTS
5 curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash
6 -
7 sudo apt install -y nodejs
8
9 # Instalar PM2 globalmente
10 sudo npm install -g pm2
11
12 # Instalar Nginx y Certbot
13 sudo apt install -y nginx certbot python3-certbot-nginx
```

Listing 15.3: Instalación del entorno en el servidor de producción

```
1 # Iniciar todos los servicios con el archivo de ecosistema
2 pm2 start ecosystem.config.js --env production
3
4 # Guardar la configuracion para reinicio automatico
5 pm2 save
6 pm2 startup
7
8 # Verificar estado de todos los procesos
9 pm2 status
```

Listing 15.4: Iniciar los microservicios con PM2

Capítulo 16

Referencia de la API REST

16.1 Convenciones Generales

- Todas las rutas usan el prefijo `/api/v1/`.
- El cuerpo de peticiones y respuestas es siempre `application/json`.
- Las rutas protegidas requieren la cabecera: `Authorization: Bearer <token>`.
- Los errores devuelven siempre `{"message": "descripcion"}`.
- Las fechas se transmiten en formato ISO 8601.

16.2 Códigos de Respuesta HTTP

Cuadro 16.1: Códigos de respuesta utilizados en la API

Código	Estado	Descripción
200	OK	Operación exitosa.
201	Created	Recurso creado exitosamente.
400	Bad Request	Parámetros inválidos o faltantes.
401	Unauthorized	Token ausente, inválido o expirado.
403	Forbidden	Token válido pero rol insuficiente.
404	Not Found	Recurso no encontrado.
409	Conflict	Conflicto (ej. documento ya registrado).
500	Internal Server Error	Error inesperado del servidor.

16.3 Endpoints por Servicio

16.3.1 Auth Service — /api/v1/auth

Cuadro 16.2: Endpoints del Auth Service

Método	Ruta	Descripción	Rol
POST	/login	Autenticar usuario y obtener JWT.	Público
POST	/usuarios	Crear nuevo usuario en el sistema.	Admin
GET	/usuarios	Listar todos los usuarios.	Admin
GET	/usuarios/:id	Obtener usuario por ID.	Admin
PUT	/usuarios/:id	Editar datos del usuario.	Admin
PATCH	/usuarios/:id/estado	Bloquear o activar usuario.	Admin
DELETE	/usuarios/:id	Eliminar usuario del sistema.	Admin
GET	/auditoria	Consultar registros de auditoría.	Admin
GET	/configuracion	Obtener tipos de incidente y roles.	Admin
POST	/configuracion/tipo	Agregar tipo de incidente.	Admin
DELETE	/configuracion/tipo/:id	Eliminar tipo de incidente.	Admin

16.3.2 Accesos Service — /api/v1/accesos

Cuadro 16.3: Endpoints del Accesos Service

Método	Ruta	Descripción	Rol
POST	/institucional	Registrar acceso de usuario institucional.	Celador, Admin
POST	/visitante	Registrar acceso de visitante externo.	Celador, Admin
GET	/	Historial completo con filtros.	Celador, Admin
GET	/hoy	Registros del día actual.	Celador, Admin
GET	/estadisticas	Estadísticas por rango de fecha.	Admin

16.3.3 Incidentes Service — /api/v1/incidentes

Cuadro 16.4: Endpoints del Incidentes Service

Método	Ruta	Descripción	Rol
POST	/	Crear reporte de incidente.	Usuario, Celador, Admin
GET	/	Todos los reportes.	Celador, Admin
GET	/mis-reportes	Reportes del usuario autenticado.	Usuario
GET	/:id	Detalle de un reporte.	Todos
PATCH	/:id/revision	Tomar el incidente en revisión.	Celador, Admin
PATCH	/:id/resolver	Marcar incidente como resuelto.	Celador, Admin
PUT	/:id	Editar cualquier campo del reporte.	Admin

16.3.4 Objetos Service — /api/v1/objetos

Cuadro 16.5: Endpoints del Objetos Service

Método	Ruta	Descripción	Rol
POST	/	Publicar objeto perdido o encontrado.	Usuario, Celador, Admin
GET	/	Listar objetos con filtros.	Todos (autenticados)
GET	/:id	Detalle de una publicación.	Todos (autenticados)
PATCH	/:id/encontrado	Marcar objeto como devuelto.	Admin
POST	/:id/comentarios	Agregar comentario a una publicación.	Todos (autenticados)
DELETE	/:id	Eliminar publicación.	Admin

Capítulo 17

Mantenimiento y Operación

17.1 Monitoreo del Sistema en Producción

```
1 # Panel interactivo con CPU y RAM en tiempo real
2 pm2 monit
3
4 # Ver logs de un servicio especifico
5 pm2 logs auth-service --lines 100
6
7 # Estado de todos los procesos
8 pm2 status
9
10 # Reiniciar un servicio especifico
11 pm2 restart incidentes-service
12
13 # Reiniciar todos los servicios
14 pm2 restart all
```

Listing 17.1: Comandos de monitoreo con PM2

17.2 Política de Respaldos

Cuadro 17.1: Política de respaldos del sistema

Tipo	Frecuencia	Descripción	Retención
Backup automático	Diario	MongoDB Atlas Continuous Backup. Punto de recuperación diario.	7 días
Backup manual	Semanal	mongodump exportado a directorio /backups/.	30 días
Snapshot VPS	Mensual	Imagen completa del servidor de producción.	3 meses

17.3 Actualizaciones del Sistema

```
1 # 1. Conectarse al servidor
2 ssh usuario@ip-del-servidor
3
4 # 2. Obtener los ultimos cambios del repositorio
5 cd /var/www/seguridad-universitaria
6 git pull origin main
7
8 # 3. Reinstalar dependencias si hubo cambios en package.json
9 cd services/incidentes-service && npm install --production &&
   cd ../..
10
11 # 4. Reconstruir el frontend React
12 cd frontend && npm run build && cd ..
13
14 # 5. Reiniciar los servicios afectados
15 pm2 restart incidentes-service
16
17 # 6. Verificar estado general
18 pm2 status && pm2 logs --lines 20
```

Listing 17.2: Proceso de actualización del sistema en producción

17.4 Renovación del Certificado SSL

Let's Encrypt emite certificados válidos por 90 días. Certbot instala automáticamente un *cron job* que renueva el certificado cuando quedan menos de 30 días de validez. No se requiere intervención manual en condiciones normales.

```
1 # Ver fecha de vencimiento del certificado actual
2 sudo certbot certificates
3
4 # Forzar renovacion manual (solo si fuera necesario)
5 sudo certbot renew --force-renewal
6 sudo systemctl reload nginx
```

Listing 17.3: Verificación y renovación manual del certificado SSL

Capítulo 18

Glosario Técnico

API Gateway

Componente central que recibe todas las peticiones del cliente, verifica la autenticación JWT y las enruta al microservicio correspondiente.

bcrypt

Algoritmo de hash criptográfico diseñado para almacenar contraseñas de forma segura. Usa un factor de coste configurable para aumentar el tiempo de cómputo y resistir ataques de fuerza bruta.

Certbot

Herramienta cliente de Let's Encrypt que automatiza la emisión, instalación y renovación de certificados SSL/TLS e integra directamente con Nginx.

CORS

Cross-Origin Resource Sharing. Mecanismo que permite o restringe las peticiones HTTP entre dominios distintos. Configurado en cada microservicio para aceptar únicamente peticiones del frontend autorizado.

JWT

JSON Web Token. Estándar RFC 7519 para transmitir información de autenticación de forma segura como objeto JSON firmado criptográficamente con HMAC-SHA256.

Let's Encrypt

Autoridad certificadora sin costo patrocinada por la Linux Foundation que emite certificados SSL/TLS de 90 días con renovación automática.

Microservicio

Componente de software independiente que implementa una única capacidad de negocio, con su propio proceso, base de datos y ciclo de despliegue.

MongoDB Atlas

Servicio gestionado de base de datos MongoDB en la nube con backups automáticos, monitoreo, escalabilidad y alta disponibilidad incorporada.

Mongoose	Biblioteca ODM (<i>Object Document Mapper</i>) para Node.js que proporciona modelado de esquemas, validación de datos y consultas sobre MongoDB.
Nginx	Servidor web y proxy inverso de alto rendimiento. En este sistema actúa como terminador SSL, enrutador de tráfico hacia los microservicios y servidor de archivos estáticos del frontend React.
ODM	<i>Object Document Mapper</i> . Patrón de abstracción que mapea documentos MongoDB a objetos JavaScript, facilitando la validación y el trabajo con esquemas.
PM2	Gestor de procesos para aplicaciones Node.js en producción. Garantiza que los servicios se reinicien automáticamente tras un fallo o reinicio del servidor.
RBAC	<i>Role-Based Access Control</i> . Modelo de control de acceso donde los permisos se asignan a roles y los usuarios heredan los permisos de su rol asignado.
REST	<i>Representational State Transfer</i> . Estilo arquitectónico para APIs que usa los métodos HTTP estándar (GET, POST, PUT, PATCH, DELETE) y recursos identificados por URLs.
Scrum	Marco de trabajo ágil para el desarrollo de software que organiza el trabajo en iteraciones cortas llamadas sprints, con roles definidos, eventos regulares y artefactos específicos.
SPA	<i>Single Page Application</i> . Aplicación web que carga una sola página HTML y actualiza el contenido dinámicamente sin recargar la página completa.
SSL/TLS	<i>Secure Sockets Layer / Transport Layer Security</i> . Protocolo criptográfico que proporciona comunicaciones seguras a través de internet mediante cifrado.
UFW	<i>Uncomplicated Firewall</i> . Interfaz simplificada para iptables en sistemas Ubuntu/Debian que gestiona las reglas de firewall del servidor.
VPS	<i>Virtual Private Server</i> . Servidor virtual privado que emula un servidor dedicado dentro de un entorno de alojamiento compartido, proporcionando recursos exclusivos.

Capítulo 19

Conclusiones y Trabajo Futuro

19.1 Conclusiones

El Sistema de Seguridad Universitaria representa una solución tecnológica integral y bien fundamentada a una problemática real y urgente que afectaba a la institución universitaria. Las principales conclusiones del proyecto son:

1. **Solución integral:** La plataforma aborda de forma cohesiva los cuatro problemas críticos identificados: ausencia de registro de accesos, inexistencia de canal de incidentes, falta de gestión de objetos perdidos y carencia de alertas en tiempo real.
2. **Arquitectura robusta:** La elección de microservicios con MongoDB garantiza que el sistema sea escalable, resiliente y mantenible a largo plazo. Un fallo en un módulo no compromete la operación del resto.
3. **Seguridad por diseño:** La implementación de JWT, bcrypt, RBAC, auditoría inmutable y HTTPS en producción garantiza que la información sensible de la comunidad universitaria esté protegida.
4. **Metodología efectiva:** El uso de Scrum permitió entregas incrementales validadas con los stakeholders en cada sprint, reduciendo el riesgo de construir funcionalidades no alineadas con las necesidades reales.
5. **Presupuesto razonable:** Con una inversión estimada de \$15.180.000 COP, el sistema ofrece un retorno de valor significativo al eliminar riesgos de seguridad cuyo costo potencial (robos, accidentes, responsabilidad legal) supera ampliamente el presupuesto del proyecto.

19.2 Trabajo Futuro — Versión 2.0

Cuadro 19.1: Funcionalidades propuestas para versiones futuras

Funcionalidad	Descripción	Prioridad
Aplicación móvil	Versión nativa iOS/Android para reportar incidentes desde el celular.	Alta
Integración biométrica	Registro de accesos mediante huella dactilar o reconocimiento facial.	Media
Integración CCTV	Vinculación de registros de acceso con timestamps de cámaras de seguridad.	Media
Múltiples campus	Soporte para instituciones con más de un campus o sede.	Media
IA de detección	Análisis de patrones de incidentes para predicción y prevención.	Baja
App de escritorio	Versión Electron para los puestos de control de celadores.	Baja
Integración ERP	Conexión con sistemas académicos para sincronización de usuarios.	Baja



Sistema de Seguridad Universitaria

Documentación Técnica Completa — v1.0

Autora: Gabriela Cordoba Gonzalez

Año: 2026

Documentación técnica interna — Todos los derechos reservados

Stack: React.js · Node.js · Express · MongoDB · Nginx · JWT · PM2

Metodología: Scrum Ágil · 5 Sprints · 210 horas · 4 desarrolladores